

Reporte de Agentic Loop

1. ANÁLISIS DE ARQUITECTURA

ANÁLISIS ARQUITECTÓNICO - Validador de Prompts y Tokens

PATRÓN ARQUITECTÓNICO: Single-File Dual-Layer Architecture

Dado el requerimiento de máximo 2 archivos, se adopta una arquitectura ultra-condensada donde cada capa (frontend/backend) reside en un único artefacto autocontenido. Este patrón es válido para herramientas internas de baja escala y alta velocidad de entrega.

CAPAS DEL SISTEMA:

1. Capa de Presentación (index.html): SPA estática sin framework JS. HTML semántico + Tailwind CSS vía CDN para estilos utilitarios. Lógica de interacción con Vanilla JS puro usando Fetch API nativa para comunicación asíncrona con el backend. Gestión de estados UI: idle, loading, error, success.

2. Capa de Procesamiento (main.py): Microservicio FastAPI con un único endpoint POST /api/analyze. Middleware CORS configurado para permitir orígenes localhost y file:// (apertura directa del HTML en navegador). Lógica de negocio: $\text{cálculo de tokens} = \text{len(words)} * 1.3$ redondeado. Simulación de latencia con `asyncio.sleep(random.uniform(0.5, 2.0))` no bloqueante.

SEGURIDAD (DevSecOps):

- CORS restrictivo: solo orígenes localhost permitidos, no wildcard en producción.
- Validación de payload en backend con Pydantic BaseModel (campo prompt obligatorio, no vacío).
- Validación defensiva en frontend antes de emitir la petición (prevención de requests vacíos).
- Sin almacenamiento de datos sensibles, sin autenticación requerida (herramienta interna local).
- Headers de seguridad básicos configurados en FastAPI.

TECNOLOGÍAS:

- Backend: Python 3.10+, FastAPI, Uvicorn, Pydantic, asyncio, random.
- Frontend: HTML5, Tailwind CSS CDN, Vanilla JavaScript ES6+, Fetch API.
- Sin base de datos, sin estado persistente, sin dependencias externas de red salvo Tailwind CDN.

FLUJO DE DATOS:

Usuario escribe prompt → Validación JS (vacío = borde rojo, abort) → Fetch POST /api/analyze → FastAPI valida Pydantic → `asyncio.sleep` simulado → Respuesta JSON {tokens, latency_ms, status} → UI renderiza 3 tarjetas de resultado.

LÓGICA DE NEGOCIO - Estado del prompt:

- 'Óptimo': `tokens_estimados <= 1000`
- 'Alerta': `tokens_estimados > 1000`

DECISIONES DE DISEÑO:

- Uvicorn como servidor ASGI para soporte nativo de `async/await`.
- Pydantic para validación automática del body de entrada.
- El frontend se puede abrir directamente con `file://` o servir desde cualquier servidor estático.
- Sin build step, sin bundler, máxima simplicidad operacional.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- main.py | Estado: APROBADO | Iteraciones: 1
- requirements.txt | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 18,432

Tokens de salida (output): 8,067

Total tokens: 26,499

Horas-hombre estimadas: ~2.8 horas

Modelo utilizado: claude-sonnet-4-6

Desglose por Agente:

Arquitecto: 1,080 input | 946 output | 1 llamadas

Tech Lead: 2,574 input | 3,524 output | 3 llamadas

Desarrollador: 6,020 input | 2,558 output | 3 llamadas

QA: 8,758 input | 1,039 output | 3 llamadas